Ενσωματωμένα Συστήματα Πραγματικού Χρόνου

Μάριος-Γεώργιος Δημόπουλος ΑΕΜ:10930

Github repository: https://github.com/Marios-Dimopoulos/Real-Time-Embedded-Systems-exercise1

1 Εισαγωγή

Η παρούσα αναφορά αναλύει την υλοποίηση του κλασικού προβλήματος Παραγωγού-Καταναλωτή (Producer-Consumer) στη γλώσσα C, με χρήση της βιβλιοθήκης νημάτων pthread. Σκοπός του προγράμματος είναι η ασφαλής (thread-safe) διαχείριση μιας κοινής μνήμης (ουράς FIFO) από πολλαπλά νήματα. Το σύστημα σχεδιάστηκε ώστε τα νήματα παραγωγοί (producers) να τοποθετούν δυναμικές εργασίες προς εκτέλεση, και τα νήματα καταναλωτές (consumers) να τις ανασύρουν και να τις εκτελούν ανεξάρτητα.

2 Δομές Δεδομένων Αρχιτεκτονική

Η αρχιτεκτονική βασίζεται σε τρεις κύριες δομές δεδομένων που εξασφαλίζουν την ευελιξία και την ασφάλεια του κώδικα:

2.1 Δομή workFunction

Αποτελεί τον πυρήνα της αφαίρεσης του συστήματος. Δεν αποθηκεύει στατικά δεδομένα, αλλά δείκτες σε συναρτήσεις (void *(*work)(void *)) και δείκτες στα ορίσματά τους (void *arg). Αυτό καθιστά την ουρά "τυφλή" ως προς το τι ακριβώς εκτελείται, επιτρέποντας την προσθήκη οποιασδήποτε μελλοντικής εργασίας χωρίς αλλαγή στον κώδικα της ουράς.

2.2 Δομή queue (Κυκλικός Buffer)

Υλοποιεί την ουρά FIFO. Χρησιμοποιεί στατικό πίνακα 10 θέσεων (QUEUESIZE) και διαχειρίζεται κυκλικά μέσω των δεικτών head και tail. Περιέχει επίσης τις απαραίτητες σημαίες full και empty.

2.3 Δομή thread_data

Χρησιμοποιείται για το πέρασμα πολλαπλών ορισμάτων (όπως ο δείκτης της ουράς και το ID του νήματος) στη συνάρτηση εκκίνησης του κάθε νήματος, λύνοντας τον περιορισμό της συνάρτησης pthread_create που δέχεται μόνο ένα όρισμα τύπου void *.

3 Μηχανισμοί Συγχρονισμού (Synchronization)

Λόγω της ταυτόχρονης πρόσβασης στην ουρά, υλοποιήθηκε αυστηρός αμοιβαίος αποκλεισμός (mutual exclusion) και συγχρονισμός μέσω των παρακάτω:

3.1 pthread_mutex_t mut

Κλειδώνει την ουρά κατά την εισαγωγή (queueAdd) και την εξαγωγή (queueDel), αποτρέποντας τις συνθήκες συναγωνισμού (race conditions)

3.2 pthread_cond_t notFull / notEmpty

Μεταβλητές συνθήκης που συντονίζουν τον ύπνο (block) και την αφύπνιση των νημάτων.

4 Ροή Εκτέλεσης

4.1 Η πλευρά του Παραγωγού

Κάθε νήμα-παραγωγός δημιουργεί μια εργασία υπολογισμού ημιτόνου (calculate_sine). Ένα θετικό στοιχείο της υλοποίησης είναι η χρήση δυναμικής μνήμης (malloc) για τη δημιουργία του ορίσματος της συνάρτησης (arg) σε κάθε επανάληψη. Αυτό διασφαλίζει ότι οι μεταβλητές δεν αλληλεπικαλύπτονται στη μνήμη, αποφεύγοντας φαινόμενα όπου όλοι οι καταναλωτές διαβάζουν την ίδια, τελευταία ενημερωμένη τιμή.

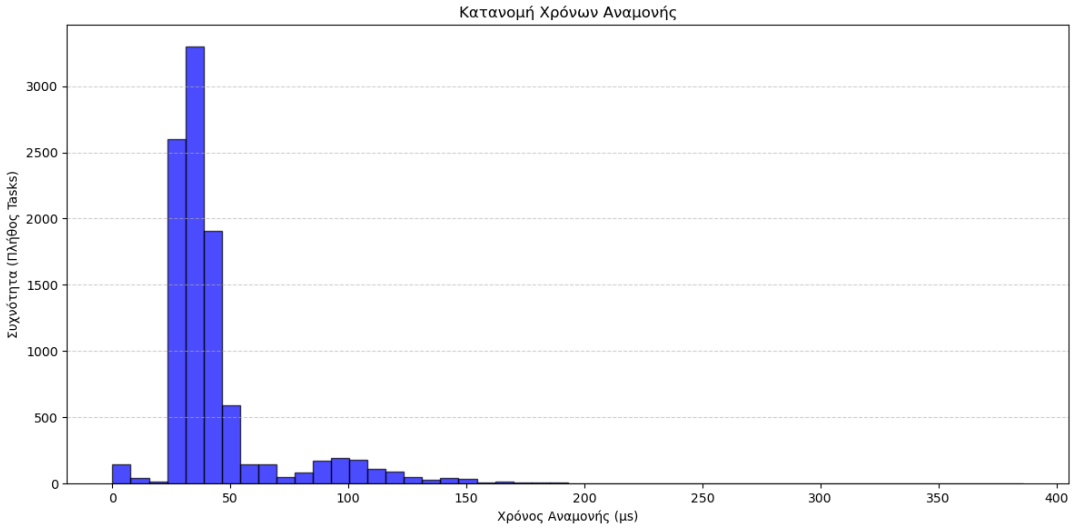
4.2 Η πλευρά του Καταναλωτή

Ο καταναλωτής κλειδώνει το mutex, παίρνει την εργασία, ξεκλειδώνει το mutex και μετά εκτελεί τη συνάρτηση. Η επιλογή να εκτελείται η συνάρτηση task.work(task.arg) εκτός του critical section αποτελεί βέλτιστη πρακτική (best practice), καθώς επιτρέπει την πραγματική παράλληλη εκτέλεση πολύπλοκων υπολογισμών, διατηρώντας το κλείδωμα της ουράς στο ελάχιστο δυνατό. Η διαχείριση μνήμης ολοκληρώνεται ορθά με την κλήση της free() εντός της calculate_sine

5 Παραδείγματα

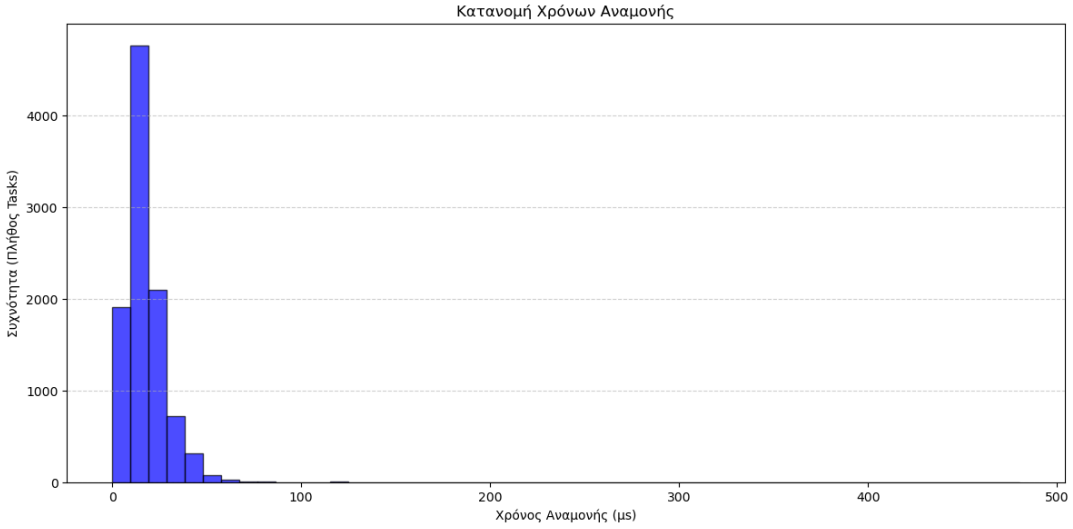
5.1 P=1, C=1

— Στατιστικά Ουράς Αναμονής — Συνολικά Tasks: 10000 / Ελάχιστος χρόνος αναμονής: 0 μs / Μέγιστος χρόνος αναμονής: 386 μs / Μέσος χρόνος αναμονής: 43.48 μs



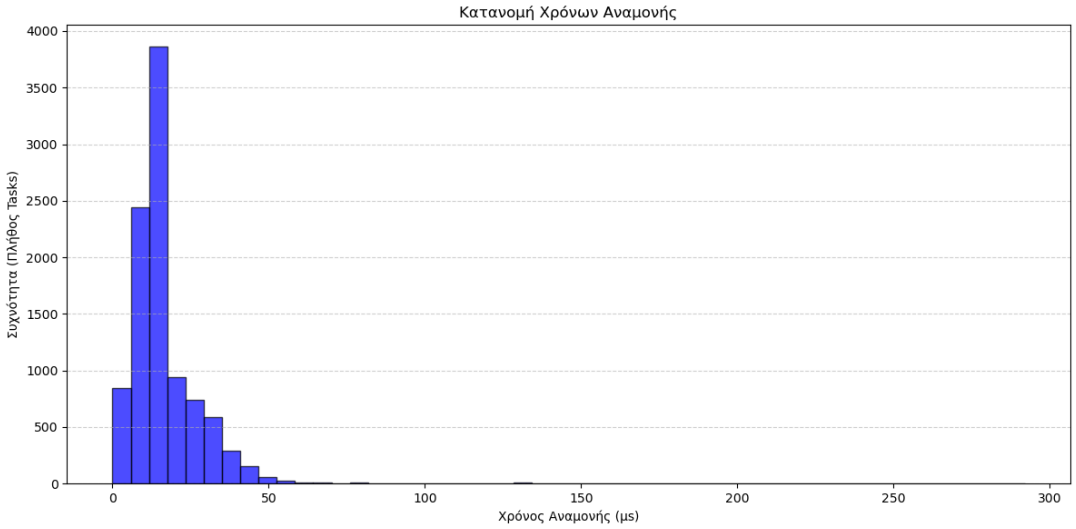
5.2 P=1, C=3

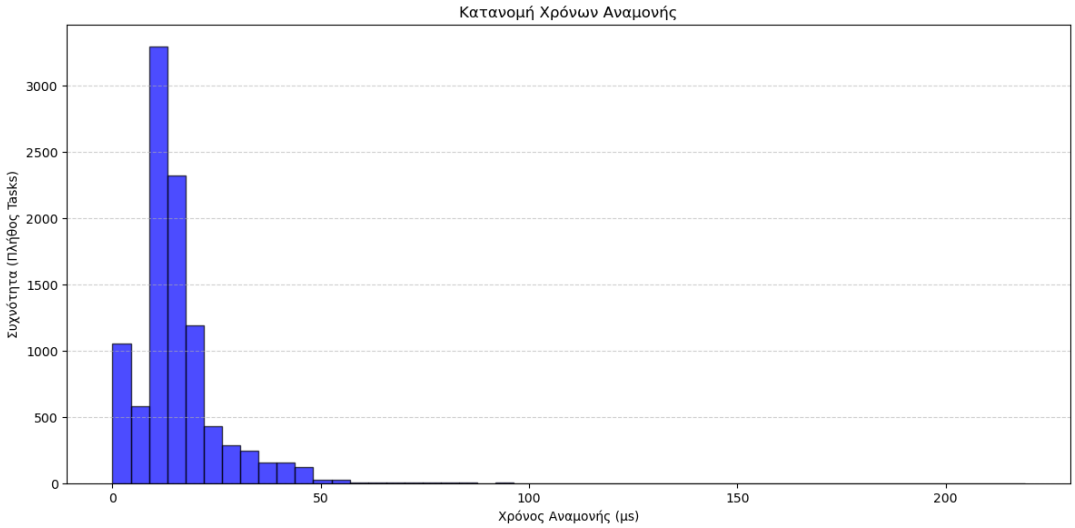
— Στατιστικά Ουράς Αναμονής — Συνολικά Tasks: 10000 / Ελάχιστος χρόνος αναμονής: 0 μs / Μέγιστος χρόνος αναμονής: 480 μs / Μέσος χρόνος αναμονής: 17.55 μs



5.3 P=1, C=5

— Στατιστικά Ουράς Αναμονής — Συνολικά Tasks: 10000 / Ελάχιστος χρόνος αναμονής: 0 μs / Μέγιστος χρόνος αναμονής: 292 μs / Μέσος χρόνος αναμονής: 16.49 μs



5.4 P=1, C=9

— Στατιστικά Ουράς Αναμονής — Συνολικά Tasks: 10000 / Ελάχιστος χρόνος αναμονής: 0 μs / Μέγιστος χρόνος αναμονής: 219 μs / Μέσος χρόνος αναμονής: 15.71 μs